

**KESESUAIAN DASAR TEORI, ARSITEKTUR, DAN BASELINE
YOLOV26 DAN DYNAMIC ATTENTION UNTUK
DETEKSI IKAN BAWAH AIR DAN KLASIFIKASI
PENYAKIT IKAN TUGAS 3**

KELOMPOK 15



Dosen Pengampu:

Prof. Dr.Eng. Fitri Utaminingrum, ST., MT.

Disusun Oleh :

JHONS JANNER SAMUEL .S	245150301111009
MUHAMMAD FADLY NUR HIDAYAT	245150300111050

**PROGRAM STUDI TEKNIK KOMPUTER
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG**

1. Pendahuluan

1.1 Latar belakang

Tugas 1 menyusun dasar teori: perkembangan YOLO, YOLO26, konsep Dynamic Attention, dan tiga dataset yang dipakai. Tugas 2 mengimplementasikan **baseline** berupa pipeline tiga model yang dilatih pada data yang sama dengan rancangan Tugas 1, tanpa DAM, sebagai titik acuan bagi YOLO26 + DAM. Tugas 3 memeriksa apakah kedua bagian tersebut saling cocok: apakah teori yang dipaparkan tercermin pada arsitektur, dan apakah baseline mengikuti arsitektur itu.

1.2 Tujuan

1. Mencocokkan dasar teori Tugas 1 dengan komponen yang dipakai pada baseline Tugas 2.
2. Mencocokkan arsitektur rancangan (YOLO26 + DAM) dengan pipeline baseline.
3. Mencocokkan dataset, konfigurasi, dan metrik evaluasi pada kedua tugas.
4. Menjelaskan perbedaan yang ada dan implikasinya untuk eksperimen berikutnya.

1.3 Ringkasan hasil pencocokan

Aspek	Status	Keterangan singkat
Alur pipeline (ikan → crop → lesi + klasifikasi)	Sesuai	Topologi sama pada Tugas 1 dan Tugas 2
Peran tiga dataset	Sesuai	Fish4Knowledge = ikan, FishDisease = lesi, Kaggle = penyakit
Skema label	Sesuai	Kelas <i>fish</i> , <i>lesion</i> , dan 7 kelas penyakit
Model dasar	Sebagian	Baseline memakai YOLO11n; rancangan memakai YOLO26
Komponen YOLO26 (dual head, STAL, MuSGD)	Sebagian / Belum	Belum ada pada baseline
Dynamic Attention Module	Belum	Baseline sengaja tanpa DAM
Metrik evaluasi	Sebagian	Sebagian metrik rencana sudah dilaporkan
Protokol split dan test independen	Sebagian	Baseline memakai split acak; test independen belum ada

2. Kesesuaian Dasar Teori dengan Baseline

2.1 Perkembangan YOLO dan pemilihan baseline

Tabel perkembangan YOLO pada Tugas 1 menempatkan YOLO11 (2024) sebagai versi yang menyeimbangkan akurasi, ukuran model, dan kemudahan deployment, serta mendukung banyak task. Hal ini sejalan dengan kebutuhan baseline: satu keluarga model dapat dipakai untuk deteksi (`yolo11n.pt`) dan klasifikasi (`yolo11n-cls.pt`) dengan antarmuka pelatihan yang sama. YOLO26 (2026) pada Tugas 1 menjadi model target penelitian, sedangkan Tugas 2 menyatakan bahwa baseline YOLO11n menjadi acuan bagi rancangan YOLO26 dengan Dynamic Attention.

Tabel 1. Komponen teori Tugas 1 dan padanannya pada baseline Tugas 2

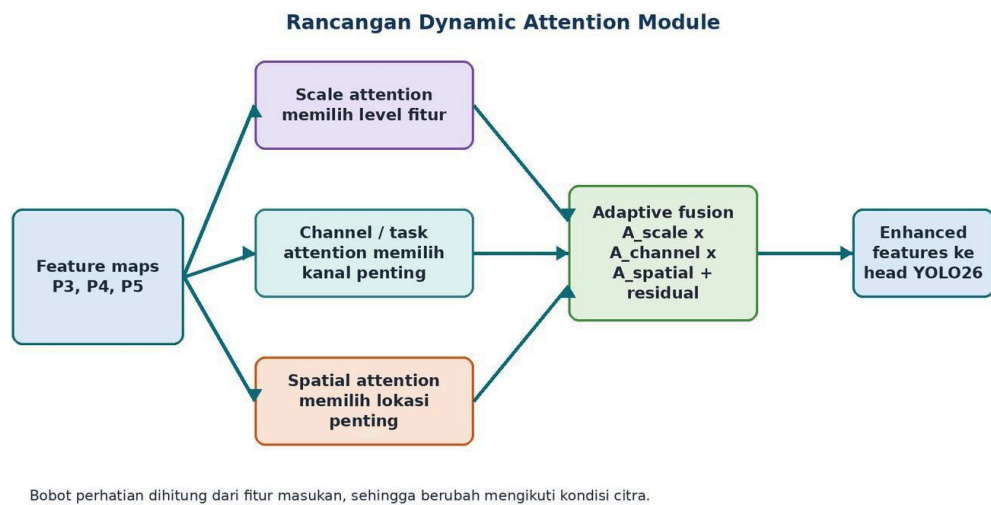
Komponen (Tugas 1)	Padanan pada baseline YOLO11n (Tugas 2)	Status
Backbone → neck (fusi multi-skala P3–P5) → head	Ada pada arsitektur YOLO secara umum, termasuk YOLO11n	Sesuai
One-to-one head, end-to-end tanpa NMS	Baseline memakai jalur deteksi dengan NMS	Sebagian
Regresi tanpa DFL	Kurva training Tugas 2 memuat <code>train/df_l_loss</code> , artinya baseline masih memakai DFL	Sebagian
STAL (label assignment objek kecil)	Tidak ada pada baseline	Belum
Progressive Loss	Tidak ada pada baseline	Belum
Optimizer MuSGD	<code>optimizer=auto</code> (bawaan Ultralytics)	Sebagian
Dynamic Attention Module (scale, channel, spatial)	Tidak ada pada baseline	Belum

2.2 Tantangan bawah air pada teori dan pada data

Tugas 1 mencatat tantangan deteksi ikan bawah air: pencahayaan tidak merata, warna berubah akibat air, gelembung, latar ramai, occlusion, serta ukuran ikan atau gejala penyakit yang kecil. Tantangan tersebut terlihat pada contoh validasi baseline (Gambar 7 dan 10 pada dokumen ini): latar karang dan perubahan warna pada citra ikan, serta lesi yang berukuran kecil pada citra flatfish.

2.3 Dynamic Attention pada teori

DAM pada Tugas 1 mengambil gagasan Dynamic Head (Dai et al., 2021): attention melalui tiga dimensi, yaitu scale (antar-level fitur), spatial (antar-lokasi), dan channel/task (di dalam kanal). DAM adalah adaptasi yang diusulkan untuk YOLO26 dan bukan komponen bawaan YOLO26.



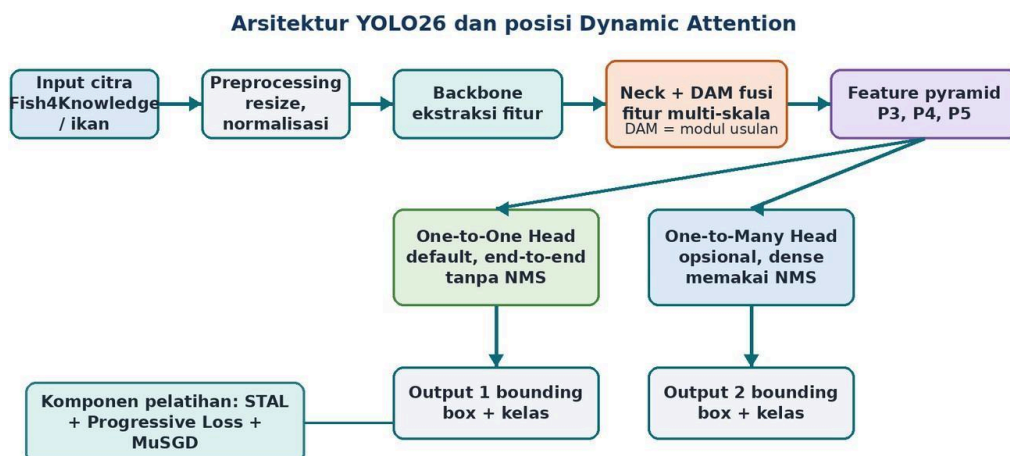
Gambar 1. Rancangan Dynamic Attention Module. Sumber: Tugas 1, Gambar 1.2.

Baseline Tugas 2 tidak memuat DAM. Peran ini sesuai: baseline diperlukan sebagai pembandingan, sehingga pengaruh modul baru dapat diukur pada tahap berikutnya.

3. Kesesuaian Arsitektur

3.1 Arsitektur YOLO26 dan posisi DAM (rancangan Tugas 1)

Pada rancangan Tugas 1, citra melewati preprocessing dan backbone, fitur digabung pada neck, DAM ditempatkan setelah fusi fitur, lalu fitur P3, P4, P5 diteruskan ke head deteksi (one-to-one sebagai jalur default, one-to-many opsional).



Keterangan: DAM adalah integrasi tambahan yang dirancang untuk menonjolkan fitur ikan dan gejala penyakit.

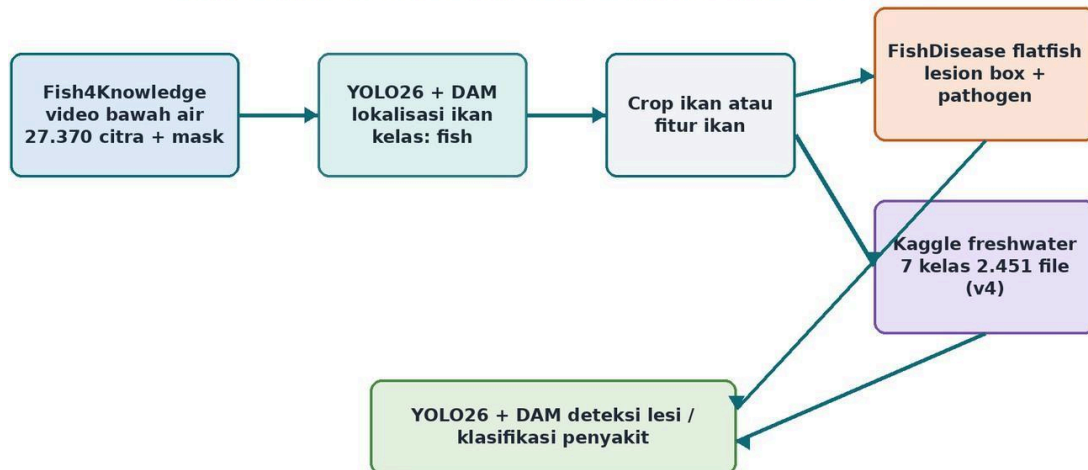
Gambar 2. Arsitektur YOLO26 dan posisi Dynamic Attention. Sumber: Tugas 1, Gambar 1.1.

Neck YOLO11n pada baseline juga menghasilkan tiga level fitur sebelum head, sehingga titik penyisipan DAM yang dirancang pada Tugas 1 tersedia pula pada baseline. Perbedaannya adalah generasi model, jenis head, dan ada tidaknya DAM.

3.2 Alur pemanfaatan dataset (rancangan) dan pipeline (implementasi)

Tugas 1 merancang alur: Fish4Knowledge untuk lokalisasi ikan, crop ikan, lalu FishDisease untuk deteksi lesi dan Kaggle untuk klasifikasi penyakit.

Alur pemanfaatan Fish4Knowledge dan dua dataset penyakit



Fish4Knowledge: domain bawah air; FishDisease: anotasi lesi; Kaggle: kelas penyakit.

Gambar 3. Alur pemanfaatan Fish4Knowledge dan dua dataset penyakit ikan. Sumber: Tugas 1, Gambar 1.6.

Tugas 2 mengimplementasikan alur yang sama dengan tiga model terpisah dan satu dashboard keluaran.



Gambar 4. Dashboard pipeline baseline: deteksi ikan (kotak kuning), jumlah lesi, dan tiga prediksi penyakit teratas. Sumber: Tugas 2, Gambar 10. Catatan: judul panel pada dashboard bertuliskan "YOLOv26 Baseline Analysis", sedangkan model yang dijalankan adalah YOLO11n.

Tabel 2. Kesesuaian tahap pipeline

Tahap	Rancangan Tugas 1	Implementasi Tugas 2	Status
1. Deteksi ikan	YOLO26 + DAM, kelas <code>fish</code>	YOLO11n detect, satu kelas <code>fish</code> → bounding box + confidence	Sebagian peran sama, model berbeda
2. Crop	Crop ikan sebagai masukan tahap berikutnya	Pemotongan <code>image[y1:y2, x1:x2]</code> per bounding box	Sesuai
3. Deteksi lesi	YOLO26 + DAM, bounding box lesi (FishDisease)	YOLO11n detect, satu kelas <code>lesion</code>	Sebagian peran sama, model berbeda
4. Klasifikasi	YOLO26-cls + DAM, 7 kelas penyakit (Kaggle)	YOLO11n-cls, 7 kelas, keluaran Top-3 + probabilitas	Sebagian peran sama, model berbeda
5. Dashboard	Tidak dirancang eksplisit	OpenCV compositing (bounding box + panel informasi)	Sesuai tambahan implementasi

3.3 Prinsip pemisahan tugas

Tugas 1 menegaskan bahwa label penyakit tingkat citra tidak boleh diklaim sebagai lokasi lesi. Prinsip ini dipenuhi pada Tugas 2: lokasi lesi dihasilkan hanya oleh detektor lesi (dilatih dengan bounding box FishDisease), sedangkan classifier hanya mengeluarkan kelas penyakit.

4. Kesesuaian Dataset dan Persiapan Data

4.1 Peran dan skema label

Tabel 3. Dataset pada rancangan dan pada baseline

Dataset	Peran (Tugas 1)	Anotasi (Tugas 1)	Pemakaian pada baseline (Tugas 2)	Status
Fish4Knowledge	Deteksi/lokalisasi ikan	Citra + mask, 23 cluster spesies	Mask → kontur terbesar → bounding box YOLO, satu kelas <code>fish</code>	Sesuai
FishDisease / FlatIMG	Deteksi lesi dan label patogen	Bounding box lesi, lokasi, patogen multi-label	Bounding box dari JSON → YOLO, satu kelas <code>lesion</code> (lokasi dan patogen belum dipakai)	Sebagian
Freshwater Fish Disease (Kaggle)	Klasifikasi penyakit tingkat citra	7 kelas, tanpa bounding box	Struktur folder kelas, 7 kelas	Sesuai



Gambar 5. Contoh foto flatfish pada dataset FishDisease/FlatIMG. Sumber: Tugas 1.

4.2 Jumlah data

Tabel 4. Jumlah data pada sumber (Tugas 1) dan yang dipakai baseline (Tugas 2)

Dataset	Sumber (Tugas 1)	Train (Tugas 2)	Val (Tugas 2)	Catatan
Fish4Knowledge	27.370 citra, 23 cluster	5.374 gambar / 9.690 objek	6.728 gambar / 2.422 objek	Baseline memakai subset; semua objek disatukan menjadi kelas <code>fish</code>
FishDisease	1.861 foto, 164 individu, 394 kasus lesi unik, 446 label lesi	929 gambar / 1.705 objek	226 gambar / 441 objek	Hanya JSON beranotasi yang dipakai; satuan hitung objek perlu disamakan (lihat 4.3)
Kaggle	250 citra per kelas $\times$ 7 kelas; ringkasan file v4: 2.451 file	250 per kelas (1.750)	100 per kelas (700)	Artefak lokal 350 per kelas

4.3 Catatan konsistensi angka

Beberapa angka pada Tabel 4 perlu dikonfirmasi terhadap data asli agar kedua laporan konsisten:

Butir	Angka	Yang perlu dikonfirmasi
Porsi val Fish4Knowledge	$6.728 / (5.374 + 6.728) \approx 55,6\%$	Skrip persiapan memindahkan 20% ke validation
Objek per gambar Fish4Knowledge	Train $9.690 / 5.374 \approx 1,80$; val $2.422 / 6.728 \approx 0,36$	Skrip menulis satu bounding box per gambar (kontur terbesar)
Objek FishDisease	2.146 objek pada Tugas 2 vs 446 label lesi pada Tugas 1	Satuan hitung (label vs bounding box), citra hasil generasi, banyak kotak per foto
Rasio split Kaggle	$250 : 100 \approx 71 : 29$	Tugas 2 menyebut rasio 80 : 20 (yang setara 280 : 70)
Duplikasi Kaggle	Tugas 2 mencatat gambar validasi berulang terhadap train	Hash konten untuk membedakan dua kemungkinan: dataset memang ± 350 citra per kelas, atau ± 250 citra unik per kelas dengan 100 salinan pada validation

Angka-angka tersebut tidak mengubah kesesuaian alur, tetapi memengaruhi cara membaca metrik (lihat Bab 6 dan 7).

4.4 Preprocessing dan pembagian data

Tabel 5. Rancangan pengolahan data (Tugas 1) dan implementasinya (Tugas 2)

Butir	Rancangan Tugas 1	Implementasi Tugas 2	Status
Fish4Knowledge → label	Image + mask → bounding box	<code>findContours</code> → kontur terbesar → <code>boundingRect</code> → label YOLO kelas 0	Sesuai
FishDisease → label	Pasangkan citra dan anotasi, konversi ke YOLO	JSON → clipping bbox → label YOLO kelas 0; nama file diberi prefix folder halaman	Sesuai
Kaggle	Periksa folder, kelas, duplikasi; split stratified	<code>splitfolders</code> .ratio 80:20, seed 42; subfolder = kelas	Sebagian
Split	Berdasarkan trajectory/sumber video, individu/sesi; test terpisah	Acak per pasangan (seed 42; FishDisease dengan peluang 20%); test belum ada	Sebagian
Citra hasil generasi FlatIMG	Ditandai terpisah saat evaluasi	Belum ada penanda	Belum
Ukuran masukan	Resize dengan rasio terjaga	Detektor 640 px, classifier 224 px	Sesuai

5. Kesesuaian Konfigurasi Training dan Metrik Evaluasi

5.1 Konfigurasi

Tabel 6. Konfigurasi eksperimen baseline (Tugas 2)

Parameter	Deteksi ikan	Deteksi lesi	Klasifikasi
Model	yolo11n.pt	yolo11n.pt	yolo11n-cls.pt
Epoch	10	10	10
Image size	640	640	224
Batch	16	16	16
Optimizer	auto	auto	auto
Augmentasi utama	mosaic, flip LR, randaugment	mosaic, flip LR, randaugment	flip LR, randaugment, erasing
Seed	0 (deterministic)	0 (deterministic); resume dari <code>last.pt</code>	0 (deterministic)

Tugas 1 mensyaratkan perbandingan yang adil: data, ukuran model, seed, anggaran pelatihan, dan aturan evaluasi yang sebanding. Konfigurasi pada Tabel 6 dapat dijadikan acuan yang sama untuk YOLO26 tanpa DAM dan dengan DAM pada tahap berikutnya.

5.2 Metrik

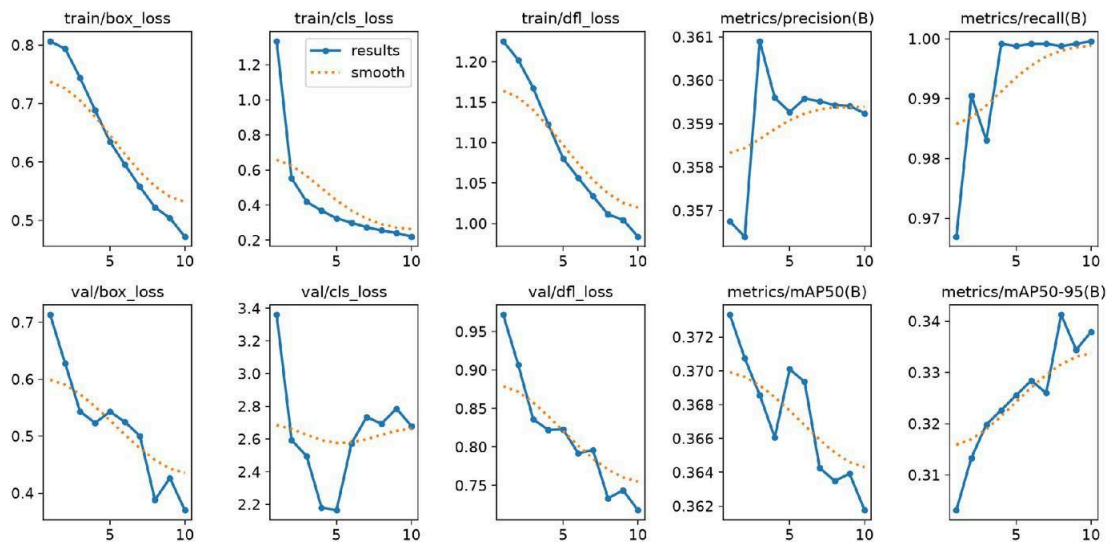
Tabel 7. Metrik yang direncanakan (Tugas 1) dan yang sudah dilaporkan (Tugas 2)

Tugas	Rencana Tugas 1	Dilaporkan Tugas 2	Status
Deteksi ikan / lesi	mAP@50-95, precision, recall, latency/FPS	Precision, recall, mAP50, mAP50-95, kurva loss, confusion matrix, kurva PR (lesi)	Sebagian latency/FPS belum diukur
Klasifikasi	Accuracy, macro precision/recall/F1, confusion matrix	Top-1, Top-5, kurva loss, confusion matrix	Sebagian macro F1 dan laporan per kelas belum ada
Patogen multi-label	Evaluasi per tipe patogen	—	Belum
Ablasi DAM	Tanpa DAM, kanal, kanal-spasial, lengkap	—	Belum

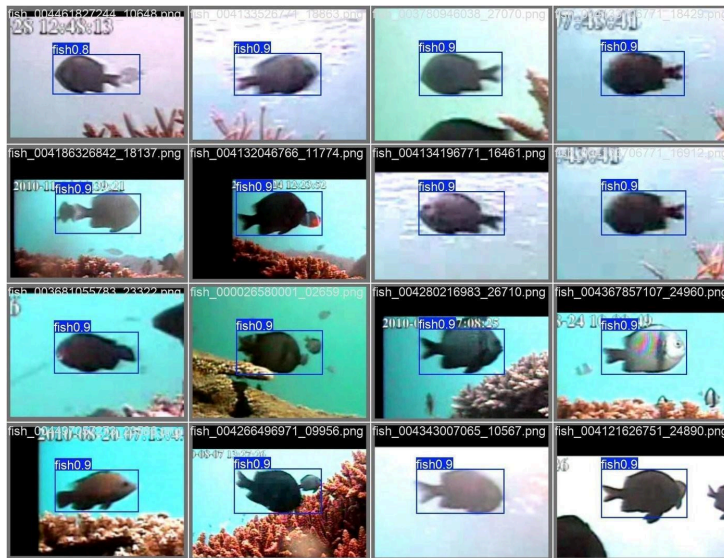
6. Hasil Baseline dan Pencocokannya dengan Rancangan

6.1 Deteksi ikan (Fish4Knowledge)

Loss box turun dari 0,8066 menjadi 0,4723 dan loss klasifikasi turun dari 1,3337 menjadi 0,2206. Recall validation sangat tinggi sejak awal (0,9670 pada epoch 1, 0,9996 pada epoch 10), sedangkan precision berada sekitar 0,36. Nilai mAP50 tertinggi terjadi pada epoch 1 (0,3733) dan nilai akhirnya 0,3618; mAP50-95 naik dari 0,3031 menjadi 0,3379 (tertinggi 0,3412 pada epoch 8).



Gambar 6. Kurva training dan metrik validasi deteksi ikan. Sumber: Tugas 2, Gambar 1.

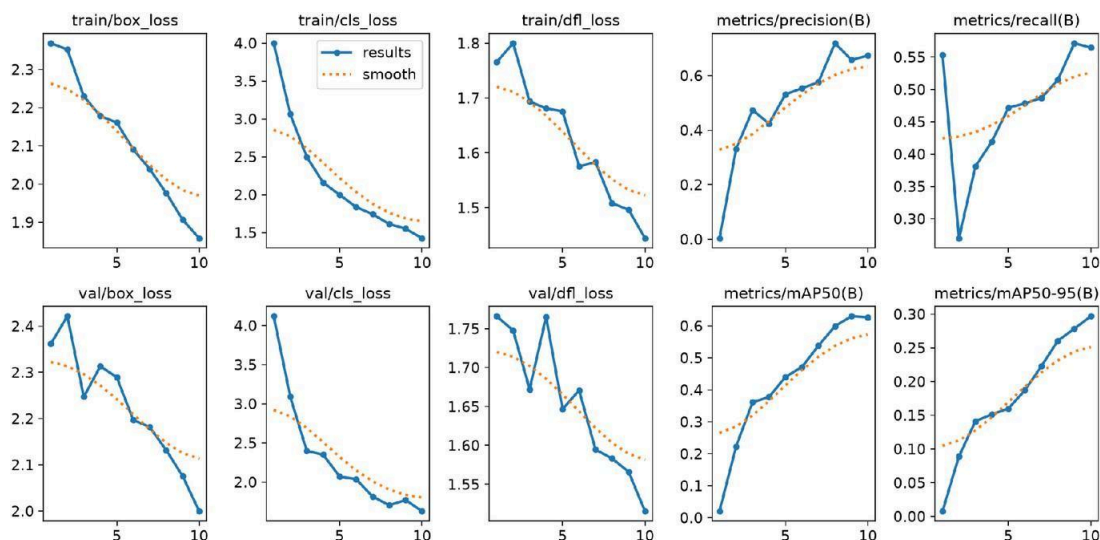


Gambar 7. Contoh prediksi deteksi ikan pada validation batch. Sumber: Tugas 2, Gambar 3.

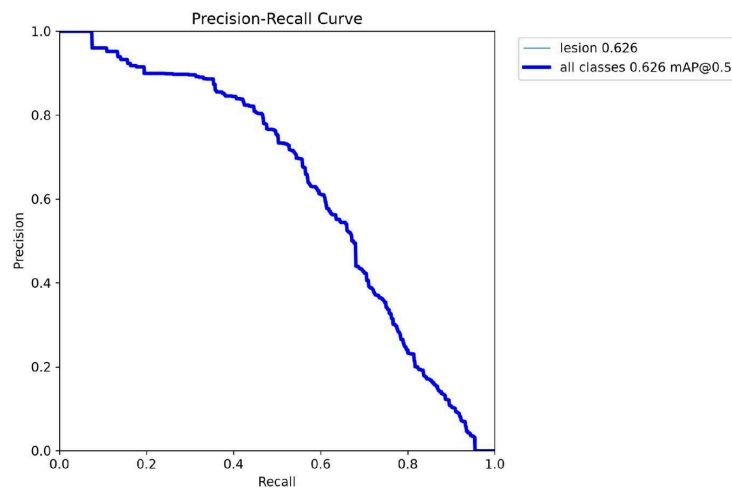
Pencocokan dengan teori: pola recall tinggi dengan precision rendah menunjukkan model menangkap hampir semua ikan tetapi banyak prediksi terhitung salah. Pada teori, spatial attention dan head one-to-one tanpa NMS diusulkan untuk memprioritaskan tubuh ikan atas latar dan mengurangi prediksi duplikat. Selain faktor model, kesesuaian pasangan gambar-label pada Fish4Knowledge (butir 4.3) ikut menentukan nilai precision, sehingga perlu diperiksa sebelum perbandingan dengan DAM.

6.2 Deteksi lesi (FishDisease)

Precision naik dari 0,0036 menjadi 0,6733, mAP50 dari 0,0201 menjadi 0,6263, dan mAP50-95 dari 0,0076 menjadi 0,2968. Recall akhir 0,5646 menunjukkan sebagian lesi masih terlewat. Jarak antara mAP50 (0,6263) dan mAP50-95 (0,2968) memperlihatkan bahwa kotak sering cukup benar pada IoU 0,50 tetapi belum konsisten pada IoU yang lebih ketat.



Gambar 8. Kurva training dan metrik validasi deteksi lesi. Sumber: Tugas 2, Gambar 4.



Gambar 9. Kurva precision-recall deteksi lesi ($mAP@0,5 = 0,626$). Sumber: Tugas 2, Gambar 5.

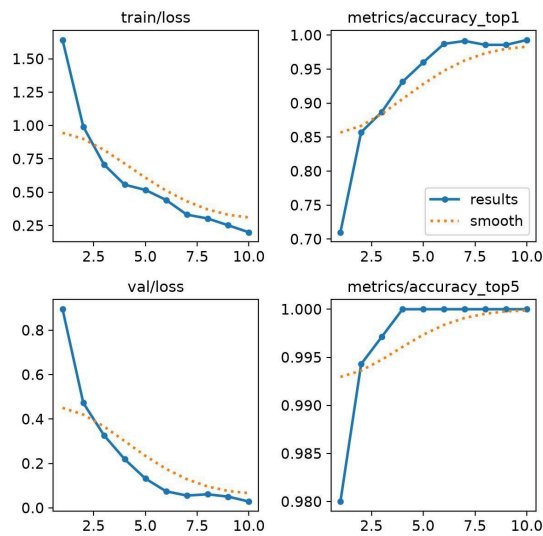


Gambar 10. Contoh prediksi lesi pada validation batch. Sumber: Tugas 2, Gambar 6.

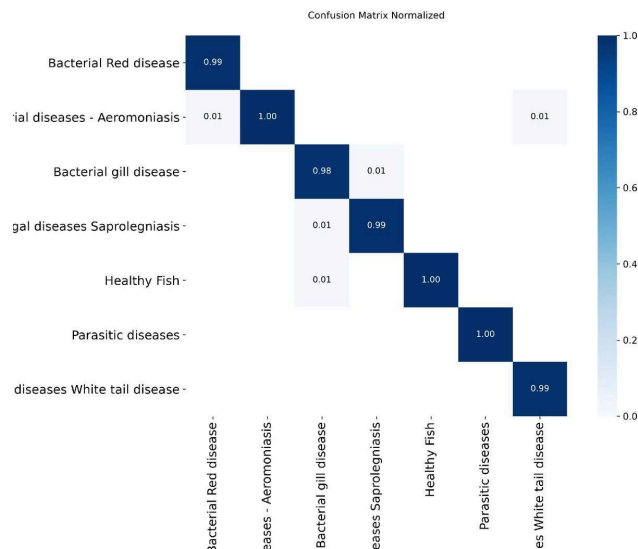
Pencocokan dengan teori: foto asli FishDisease berukuran 2048×1536 piksel, sedangkan detektor dilatih pada 640 piksel, sehingga lesi kecil menjadi lebih kecil lagi. Tugas 1 menyebut gejala penyakit yang berukuran kecil sebagai tantangan, dan mekanisme yang relevan adalah STAL (mempertahankan label positif untuk objek kecil), regresi tanpa DFL, serta scale dan spatial attention. Kurva pada Gambar 8 masih naik pada epoch 10, sehingga sebagian peningkatan dapat pula berasal dari training yang lebih lama.

6.3 Klasifikasi penyakit (Kaggle)

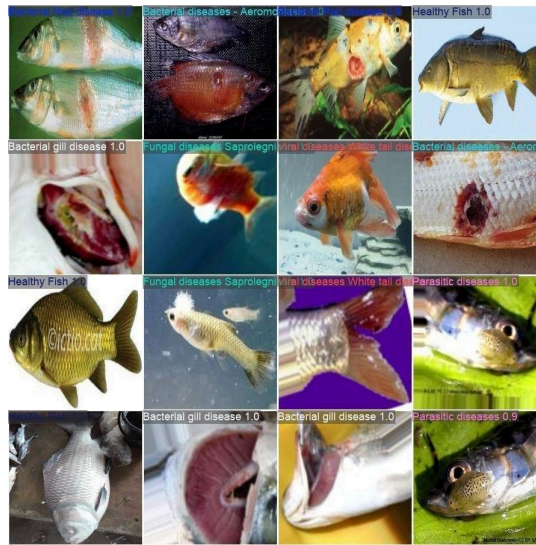
Loss train turun dari 1,6408 menjadi 0,1996 dan loss validation dari 0,8945 menjadi 0,0285. Top-1 naik dari 71,00% menjadi 99,29%, dan Top-5 mencapai 100% mulai epoch 4. Nilai diagonal confusion matrix berkisar 0,98 sampai 1,00.



Gambar 11. Kurva loss dan akurasi klasifikasi penyakit. Sumber: Tugas 2, Gambar 7.



Gambar 12. Confusion matrix ternormalisasi klasifikasi penyakit. Sumber: Tugas 2, Gambar 8.



Gambar 13. Contoh prediksi kelas penyakit pada validation batch. Sumber: Tugas 2, Gambar 9.

Pencocokan dengan teori: pada Tugas 1, klasifikasi penyakit dirancang berbasis citra dengan YOLO26-cla dan channel/spatial attention untuk warna dan tekstur. Nilai Top-1 yang tinggi pada validasi perlu dibaca bersama catatan bahwa gambar validasi berulang terhadap train (butir 4.3). Pada dashboard (Gambar 4), Top-3 untuk satu crop ikan karang hanya 23,5%, 20,7%, dan 20,2%, yaitu probabilitas yang hampir merata. Perbedaan ini wajar karena confidence satu gambar dan Top-1 validasi dihitung pada konteks data dan prosedur yang berbeda, dan karena classifier dilatih pada citra Kaggle tetapi diuji pada crop dari detektor ikan bawah air.

6.4 Ringkasan hasil baseline

Tabel 8. Hasil baseline pada epoch 1 dan epoch 10

Model	Metrik	Epoch 1	Epoch 10
Deteksi ikan	Precision	0,3568	0,3592
	Recall	0,9670	0,9996
	mAP50	0,3733	0,3618
	mAP50-95	0,3031	0,3379
Deteksi lesi	Precision	0,0036	0,6733
	Recall	0,5533	0,5646
	mAP50	0,0201	0,6263
	mAP50-95	0,0076	0,2968
Klasifikasi	Top-1	71,00%	99,29%
	Top-5	98,00%	100,00%

7. Pencocokan Teori dengan Temuan Baseline

Tabel 9. Temuan baseline, mekanisme teori yang relevan, dan hipotesis untuk tahap berikutnya

#	Temuan baseline	Mekanisme Tugas 1 yang relevan	Hipotesis yang dapat diuji
1	Ikan: recall ≈ 1 , precision $\approx 0,36$	Spatial attention (utamakan tubuh ikan atas latar); head one-to-one tanpa NMS	Setelah pasangan gambar-label diperiksa, YOLO26 + DAM menaikkan precision pada recall yang setara
2	Lesi: mAP50 0,626 vs mAP50-95 0,297; recall 0,565	STAL untuk objek kecil; regresi tanpa DFL; scale dan spatial attention	DAM menaikkan recall dan mAP50-95 lesi, dengan epoch dan imgsz yang sama pada pembandingan
3	Klasifikasi: Top-1 tinggi, keluaran dashboard tidak yakin pada crop ikan karang	Channel attention (warna, tekstur); classifier dilatih pada crop	Pada validasi bebas duplikat, YOLO26-clas dengan DAM memperbaiki Top-1 dan macro F1
4	Kecepatan belum diukur	Jalur one-to-one tanpa NMS	Latency end-to-end YOLO26 lebih rendah daripada YOLO11 + NMS pada perangkat dan batch yang sama

Kehati-hatian klaim. Baris pada Tabel 9 adalah hipotesis, bukan hasil. Teori pada Tugas 1 adalah rujukan konseptual; manfaat DAM pada tugas ikan harus dibuktikan secara eksperimen dengan pembandingan YOLO26 tanpa DAM. Domain pelatihan detektor lesi (foto flatfish di laboratorium) dan classifier (citra Kaggle) juga berbeda dari crop ikan karang pada tahap inferensi, sehingga dashboard baseline berfungsi sebagai demonstrasi keluaran sistem, bukan bukti ketepatan diagnosis.

8. Rencana Tahap Berikutnya

8.1 Prasyarat data

1. Hash konten seluruh gambar; bersihkan duplikat (khususnya Kaggle).
2. Periksa pasangan gambar-label Fish4Knowledge dan jumlah bounding box FishDisease.
3. Split berkelompok (trajectory/sumber video, individu/sesi) dan sisihkan test independen.
4. Tandai citra hasil generasi FlatIMG.
5. Simpan manifest (nama berkas, label, kelompok, split, hash).

8.2 Matriks eksperimen

Tabel 10. Rencana eksperimen YOLO26 + DAM

Kode	Model	DAM	Fungsi
B0	YOLO11n / YOLO11n-cls (dilatih ulang pada data bersih)	Tidak	Baseline Tugas 2 yang diperbarui
B1	YOLO26n / YOLO26n-cls	Tidak	Pembandingan langsung untuk mengisolasi efek DAM
D1	YOLO26n	Kanal saja	Ablasi
D2	YOLO26n	Kanal + spasial	Ablasi
D3	YOLO26n	Lengkap (scale + channel + spatial)	Model usulan
C1 -C 2	YOLO26n-cls	Kanal / kanal + spasial	Ablasi klasifikasi (tanpa scale attention karena model - cls tidak memiliki neck multi-skala; perlu dikonfirmasi pada YAML model)

Kontrol yang sama: data dan split, ukuran model, seed, epoch, imgsz, batch, augmentasi, optimizer, dan aturan evaluasi. **Metrik:** precision, recall, mAP50, mAP50-95, latency/FPS (deteksi); accuracy, macro P/R/F1, laporan per kelas, confusion matrix (klasifikasi).

9. Revisi Implementasi Menjadi YOLO26

Revisi pada bab ini menjelaskan kode lokal yang sekarang menunjuk checkpoint YOLO26. Revisi tersebut belum boleh dibaca sebagai hasil peningkatan performa, karena Dynamic Attention belum ditambahkan dan training fine-tuning YOLO26 10 epoch belum menghasilkan artefak best.pt yang dilaporkan sebagai hasil final.

9.1 Tujuan revisi kode

Tujuan utama revisi adalah menyamakan model yang dipanggil oleh kode dengan model target pada rancangan Tugas 1. Dua task deteksi menggunakan yolo26n.pt, sedangkan task klasifikasi menggunakan yolo26n-cls.pt. Alur sistem tidak diubah: gambar penuh dipakai untuk deteksi ikan, bounding box ikan dipotong, crop digunakan untuk deteksi lesi dan klasifikasi penyakit, lalu hasilnya dapat divisualisasikan pada dashboard.

Tabel 11. Sebelum dan Sesudah lalu Dampak dari revisi

Bagian	Sebelum revisi	Sesudah revisi	Dampak
Detector ikan	`yolo11n.pt`	`yolo26n.pt`	Baseline mengikuti generasi model target
Detector lesi	`yolo11n.pt`	`yolo26n.pt`	Task deteksi tetap memakai satu kelas `lesion`
Classifier	`yolo11n-cls.pt`	`yolo26n-cls.pt`	Task klasifikasi memakai keluarga YOLO26
Seed	Artefak lama dapat memakai seed 0	`seed=42`	Split/training lebih mudah direproduksi

Resume	Riwayat lama dapat melanjutkan `last.pt`	`resume=False`	Run baru tidak mengambil checkpoint lama
Dynamic Attention	Belum ada	Tetap belum ada	Baseline YOLO26 tetap menjadi pembanding tanpa DAM
Evaluasi awal	Hasil fine-tuning YOLO11	YOLO26 pretrained/zero-shot	Status hasil dipisahkan agar tidak menyesatkan

9.2 Struktur file yang digunakan

Tabel 12. Struktur File

File	Fungsi pada revisi
`scripts/3_training_all.py`	Melatih dua detector YOLO26n dan satu classifier YOLO26n-cls
`scripts/evaluate_yolo26_baseline.py`	Mengevaluasi checkpoint pretrained YOLO26 pada validation set lokal
`all_in_one_yolocovis.py`	Menyediakan preprocessing, YAML, training, evaluasi, dan pipeline terpadu
`scripts/4_tes_pipeline_final.py`	Menjalankan inferensi berantai dan membuat dashboard
`data_ikan.yaml`	Mendefinisikan dataset deteksi ikan dan kelas `fish`
`data_lesi.yaml`	Mendefinisikan dataset deteksi lesi dan kelas `lesion`
`reports/yolo26_baseline_metrics.json`	Menyimpan metrik evaluasi YOLO26 yang sudah tersedia

10. Kode Training YOLO26 dan Penjelasannya

10.1 Program training lengkap

Berikut adalah isi `scripts/3_training_all.py` yang menjadi entry point training. Kode ini tidak menambahkan DAM. Ia hanya mengganti checkpoint model dan membuat konfigurasi training lebih eksplisit.

```
from ultralytics import YOLO

EPOCHS = 10
SEED = 42
PROJECT = "Runs_Baseline"

print("--- 1. TRAINING DETEKSI IKAN ---")
model_ikan = YOLO("yolo26n.pt")
model_ikan.train(
    data="D:/yolocovis/data_ikan.yaml", epochs=EPOCHS, imgsz=640,
    project=PROJECT, name="model_ikan", seed=SEED,
    deterministic=True, resume=False
)
```

```
print("--- 2. TRAINING DETEKSI LESI (LUKA) ---")
model_lesi = YOLO("yolo26n.pt")
model_lesi.train(
    data="D:/yolocomvis/data_lesi.yaml", epochs=EPOCHS, imgsz=640,
    project=PROJECT, name="model_lesi", seed=SEED,
    deterministic=True, resume=False
)

print("--- 3. TRAINING KLASIFIKASI PENYAKIT (KAGGLE) ---")
model_penyakit = YOLO("yolo26n-cls.pt")
model_penyakit.train(
    data="D:/yolocomvis/datasets/freshwater_kaggle",
    epochs=EPOCHS, imgsz=224, project=PROJECT,
    name="model_penyakit", seed=SEED,
    deterministic=True, resume=False
)
```

10.2 Penjelasan setiap bagian

1. from ultralytics import YOLO memanggil antarmuka resmi Ultralytics untuk task deteksi dan klasifikasi.
2. EPOCHS = 10 mempertahankan anggaran pelatihan yang sama dengan baseline Tugas 2 agar perubahan model tidak sekaligus mengubah jumlah epoch.
3. SEED = 42 digunakan sebagai parameter eksplisit. Nilai ini harus dicatat bersama versi dataset dan perangkat karena seed saja tidak menjamin hasil identik pada hardware berbeda.
4. YOLO("yolo26n.pt") memuat checkpoint pretrained YOLO26 ukuran nano untuk deteksi bounding box. Checkpoint yang sama dapat digunakan untuk deteksi ikan dan lesi karena dataset YAML menentukan nama kelas serta labelnya.
5. YOLO("yolo26n-cls.pt") memuat varian klasifikasi. Berbeda dari detector, input data classifier adalah folder yang berisi subfolder nama kelas.
6. imgsz=640 dipakai pada deteksi dan imgsz=224 pada klasifikasi, sesuai karakteristik task serta konfigurasi baseline sebelumnya.
7. deterministic=True meminta proses yang lebih deterministik. resume=False mencegah training baru meneruskan last.pt dari run lain.
8. project dan name menentukan lokasi artefak, yaitu folder Runs_Baseline/model_ikan, model_lesi, dan model_penyakit.

Konfigurasi ini baru mengubah checkpoint model yang dipanggil oleh training. Komponen teori seperti DAM, ablasi kanal-spasial, dan perbandingan YOLO26 tanpa DAM versus YOLO26 dengan DAM belum tercermin di kode ini.

11. Kode Evaluasi dan Interpretasi Metrik

11.1 Program evaluasi pretrained YOLO26

```
from __future__ import annotations

import json
from pathlib import Path
from ultralytics import YOLO

ROOT = Path(__file__).resolve().parents[1]
OUTPUT = ROOT / "reports" / "yolo26_baseline_metrics.json"
VALIDATION_FRACTION = 1.0

def evaluate() -> None:
    OUTPUT.parent.mkdir(parents=True, exist_ok=True)
    metrics = {}
    for name, model_name, data, image_size in (
        ("fish_detection", "yolo26n.pt", ROOT / "data_ikan.yaml", 640),
        ("lesion_detection", "yolo26n.pt", ROOT / "data_lesi.yaml",
640),
        ("disease_classification", "yolo26n-cls.pt",
        ROOT / "datasets" / "freshwater_kaggle", 224),
    ):
        model = YOLO(model_name)
        result = model.val(
            data=str(data), imgsz=image_size, batch=32, workers=0,
            fraction=VALIDATION_FRACTION, plots=True, verbose=False,
        )
        if name == "disease_classification":
            metrics[name] = {
                "status": "zero-shot-pretrained-full-validation",
                "model": model_name, "fraction": VALIDATION_FRACTION,
                "top1": float(result.top1), "top5": float(result.top5),
            }
        else:
            box = result.box
            metrics[name] = {
                "status": "zero-shot-pretrained-full-validation",
                "model": model_name, "fraction": VALIDATION_FRACTION,
                "precision": float(box.mp), "recall": float(box.mr),
                "map50": float(box.map50), "map50_95": float(box.map),
            }
    OUTPUT.write_text(json.dumps(metrics, indent=2), encoding="utf-8")

if __name__ == "__main__":
    evaluate()
```

11.2 Penjelasan evaluasi

1. ROOT membuat path relatif terhadap lokasi script, sehingga perintah dapat dijalankan dari

direktori kerja berbeda.

- fraction=1.0 berarti seluruh validation set digunakan, bukan sampel kecil.
- Detector mengambil precision, recall, map50, dan map50_95 dari objek result.box.
- Classifier mengambil top1 dan top5 dari hasil validasi klasifikasi.
- Field status sengaja ditulis sebagai zero-shot-pretrained-full-validation. Artinya checkpoint YOLO26 umum dievaluasi langsung pada data proyek dan belum dinyatakan sebagai checkpoint hasil fine-tuning lokal.
- JSON dipakai sebagai sumber terstruktur agar tabel laporan dapat dibuat ulang tanpa menyalin angka secara manual.

11.3 Hasil YOLO26 yang tersedia

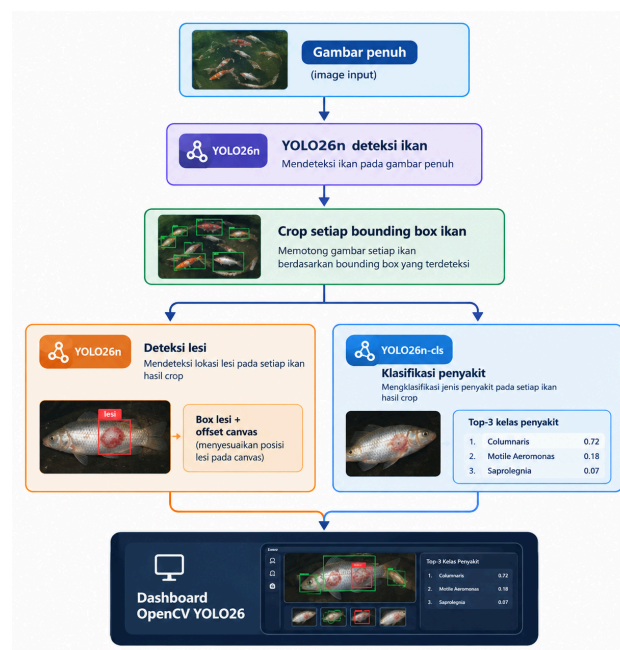
Tabel 13. Hasil Yolo26

Task	Model	Precision/Top-1	Recall/Top-5	mAP50	mAP50-95
Deteksi ikan	`yolo26n.pt`	0,1971	0,3551	0,1531	0,1024
Deteksi lesi	`yolo26n.pt`	0,0574	0,0522	0,0041	0,0017
Klasifikasi penyakit	`yolo26n-cls.pt`	3,57%	8,57%	-	-

Hasil tersebut adalah pemeriksaan zero-shot, bukan hasil training lokal 10 epoch. Nilai rendah pada deteksi lesi dan klasifikasi konsisten dengan dugaan domain gap: checkpoint pretrained umum belum disesuaikan dengan citra FishDisease dan tujuh kelas penyakit lokal. Angka ini tidak boleh dibandingkan langsung sebagai eksperimen adil dengan angka fine-tuning YOLO11 pada laporan lama.

12. Kode Pipeline Inferensi YOLO26

12.1 Urutan program



Gambar 14. Diagram urutan program

12.2 Potongan kode inferensi inti

```
deteksi_ikan = YOLO(path_ikan)
deteksi_lesi = YOLO(path_lesi)
klasifikasi = YOLO(path_penyakit)

hasil_ikan = deteksi_ikan(gambar)[0]
for kotak in hasil_ikan.bboxes:
    x1, y1, x2, y2 = map(int, kotak.xyxy[0])
    potongan_ikan = gambar[y1:y2, x1:x2]

hasil_lesi = deteksi_lesi(potongan_ikan)[0]
for luka in hasil_lesi.bboxes:
    lx1, ly1, lx2, ly2 = map(int, luka.xyxy[0])
    cv2.rectangle(canvas, (x1 + lx1, y1 + ly1),
                  (x1 + lx2, y1 + ly2), COLOR_BOX_LESI, 2)

hasil_klasifikasi = klasifikasi(potongan_ikan)[0]
top5_indices = hasil_klasifikasi.probs.top5
top_classes = [klasifikasi.names[i] for i in top5_indices]
top_probs = [float(hasil_klasifikasi.probs.data[i]) * 100
              for i in top5_indices]
```

Detektor ikan bekerja pada koordinat gambar penuh. Karena detector lesi bekerja pada crop, koordinat lesi harus ditambah offset (x1, y1) agar kotak kembali tepat pada canvas asli. Classifier tidak menghasilkan lokasi; ia hanya menghasilkan probabilitas kelas pada crop. Pemisahan ini mempertahankan prinsip Tugas 1 bahwa label penyakit tingkat citra tidak boleh disebut sebagai lokasi lesi.

12.3 Batasan implementasi dashboard

Dashboard menggunakan checkpoint hasil training jika folder Runs_Baseline tersedia. Jika checkpoint fine-tuning belum ada, script dashboard tidak boleh diam-diam menggantinya dengan angka zero-shot. Status model harus ditulis di caption atau catatan eksperimen. Dashboard adalah demonstrasi alur inferensi dan bukan bukti akurasi diagnosis.

13. Perbedaan Sebelum dan Sesudah Revisi

Tabel 14. Perbedaan sebelum dan sesudah revisi

Aspek	Baseline laporan lama	Revisi kode YOLO26
Model deteksi	YOLO11n	YOLO26n
Model klasifikasi	YOLO11n-cls	YOLO26n-cls
Data dan alur	Fish4Knowledge -> crop -> FishDisease/Kaggle	Dipertahankan

Input detector	640	640
Input classifier	224	224
Epoch target	10	10
Seed yang ditulis	Artefak lama dapat berbeda	42
Resume checkpoint	Salah satu run lama tercatat resume	`resume=False`
DAM	Tidak ada	Tetap tidak ada
Hasil yang tersedia	Fine-tuning YOLO11	YOLO26 zero-shot/full validation
Status perbandingan	Baseline historis	Pemeriksaan awal kompatibilitas YOLO26

Perubahan model tidak otomatis membuat perbandingan menjadi adil. Perbandingan YOLO11 fine-tuned dengan YOLO26 zero-shot masih dipengaruhi status training yang berbeda. Eksperimen final harus melatih YOLO26 dan YOLO11 dengan split, seed, epoch, augmentasi, dan aturan evaluasi yang dikunci.

14. Prosedur Menjalankan Revisi dan Rencana Hasil Final

Urutan yang direkomendasikan adalah sebagai berikut:

1. Pastikan environment Python aktif dan paket ultralytics, opencv-python, numpy tersedia.
2. Periksa data_ikan.yaml, data_lesi.yaml, folder datasets/freshwater_kaggle, dan pasangan gambar-label.
3. Jalankan training pada GPU atau Colab agar target 10 epoch dapat selesai. Simpan args.yaml, results.csv, best.pt, dan log perangkat.
4. Jalankan evaluasi pada checkpoint best.pt, bukan checkpoint pretrained, dengan validation dan test independen yang sudah dibekukan.
5. Catat precision, recall, mAP50, mAP50-95, Top-1, Top-5, macro F1, confusion matrix, dan latency/FPS.
6. Perbarui tabel hasil hanya setelah status checkpoint dan split diverifikasi.
7. Setelah baseline YOLO26 tanpa DAM stabil, implementasikan D1-D3 dan gunakan baseline tersebut sebagai kontrol.

Hasil final yang diharapkan dari revisi bukan sekadar pergantian nama file model. Hasil final harus menunjukkan apakah YOLO26 sudah berhasil di-fine-tune pada domain ikan, apakah kinerja meningkat dibanding baseline historis pada protokol yang setara, dan apakah DAM memberikan kontribusi yang terukur. Sampai eksperimen itu dilakukan, klaim yang sah hanyalah bahwa kode sudah diarahkan ke YOLO26 dan checkpoint pretrained sudah berhasil dievaluasi.

15. Kesimpulan

1. **Arsitektur sesuai.** Alur ikan → crop → deteksi lesi dan klasifikasi → dashboard pada Tugas 2 mengikuti rancangan Tugas 1, termasuk pembagian peran tiga dataset dan skema labelnya.
2. **Teori dan baseline saling melengkapi.** Baseline YOLO11n berfungsi sebagai acuan pipeline; komponen khas YOLO26 (dual head, STAL, MuSGD) dan DAM sengaja belum ada dan menjadi tahap berikutnya.

3. **Perbedaan utama** terletak pada generasi model (YOLO11 vs YOLO26) dan pada protokol data (split acak, belum ada test independen), sehingga pembandingan YOLO26 tanpa DAM diperlukan untuk menilai kontribusi DAM.
4. **Hasil baseline** menunjukkan pola yang dapat dihubungkan dengan teori: recall tinggi tetapi precision rendah pada ikan, jarak mAP50 dan mAP50-95 pada lesi, dan Top-1 tinggi pada klasifikasi. Hubungan ini masih berupa hipotesis.
5. **Langkah berikut:** konfirmasi angka data, bersihkan dan bagi ulang data, latih B1 dan varian DAM dengan kontrol yang sama, lalu evaluasi pada test independen.

Keterbatasan dan Verifikasi Lanjutan

Dokumen ini membandingkan dua laporan tertulis (Tugas 1 dan Tugas 2); kode, log training, dan data mentah tidak diperiksa ulang. Hal-hal berikut belum terverifikasi dan menjadi bagian dari rencana tahap berikutnya:

- Jumlah citra unik pada dataset Kaggle (melalui hash konten), serta penyebab selisih 250 vs 350 citra per kelas.
- Jumlah gambar dan label Fish4Knowledge pada folder `images/` dan `labels/` (porsi validation dan objek per gambar).
- Jumlah bounding box pada JSON FishDisease dibanding 446 label lesi pada Tugas 1.
- Optimizer dan learning rate efektif pada log training (`optimizer=auto` pada Tabel 6).
- Struktur model YOLO26-cls (ada atau tidaknya neck multi-skala) untuk menentukan komponen DAM yang dapat diterapkan pada klasifikasi.
- Baseline berasal dari satu run, 10 epoch, satu seed; variansi antar-run belum diketahui, dan belum ada eksperimen YOLO26 maupun DAM

Daftar Pustaka

- Kelompok 15. *YOLOv26 + Dynamic Attention untuk Deteksi Ikan Bawah Air dan Klasifikasi Penyakit Ikan* (Tugas 1).
- Kelompok 15. *YOLOv26 dan Dynamic Attention untuk Deteksi Ikan Bawah Air dan Klasifikasi Penyakit Ikan* Tugas 2 (baseline; repositori <https://github.com/F4-dly/comviskan>).
- Dai, X., Chen, Y., Xiao, B., Chen, D., Liu, M., Yuan, L. and Zhang, L. (2021) 'Dynamic Head: Unifying Object Detection Heads With Attentions', *CVPR*, pp. 7373–7382. <https://doi.org/10.1109/CVPR46437.2021.00727>
- Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F.C. and Kalfaoglu, M.E. (2026) 'Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models', arXiv:2606.03748. <https://doi.org/10.48550/arXiv.2606.03748>
- Ultralytics (2024) *Ultralytics YOLO11*. <https://docs.ultralytics.com/models/yolo11/>
- Ultralytics (2026) *Ultralytics YOLO26*. <https://docs.ultralytics.com/models/yolo26/>
- Hwang, S.-B., Kim, H.-Y., Heo, C.-Y., Jeong, H.-Y., Jung, S.-J. and Cho, Y.-J. (2025) 'Flatfish lesion detection based on part segmentation approach and lesion image generation', *Journal of the World Aquaculture Society*, 56(3), e70031. <https://doi.org/10.1111/jwas.70031>
- Biswas, S., Muduli, D., Islam, M.A., Kanade, A.S., Zamani, A.T., Kanade, S.P. and Parveen, N. (2024) 'Empirical Evaluation of Deep Learning Techniques for Fish Disease Detection in Aquaculture Systems', *IEEE Access*, 12, pp. 176136–176154. <https://doi.org/10.1109/ACCESS.2024.3504283>
- Fish4Knowledge. *Fish Recognition Ground-Truth data*. University of Edinburgh. <https://homepages.inf.ed.ac.uk/rbf/Fish4Knowledge/GROUNDTRUTH/RECOG/>
- Biswas, S. *Freshwater Fish Disease Aquaculture in South Asia* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/subirbiswas19/freshwater-fish-disease-aquaculture-in-south-asia/data>